LiteGrip

软件开发手册

NEXFORM ROBOTICS

版本：V1.2.0　　日期：2026 年 11 月 5 日

修订记录

版本	修订时间	修订记录
V1.2.0	2026-11-05	按实际发货配置修订硬件清单与物理层说明，终端电阻改为夹爪内置
V1.1.2	2026-11-03	移除正文目录，精简修订记录
V1.1.0	2026-10-22	面向终端用户重构全文，合并 SDK 与生态章节
V1.0.0	2026-09-24	新建 LiteGrip 软件开发手册框架

目录

1 前言 1
1.1 软件组成与依赖 1
1.2 单位说明 2
1.3 安全须知 2
1.3.1 简介 2
1.3.2 安全警示标志 2
1.3.3 安全注意事项（软件侧） 3
1.3.4 责任及规范 3
1.3.5 风险评估 3

2 SDK 使用指南 4
2.1 安装 SDK 4
2.2 配置 CAN 接口 4
2.3 接 24 V 电源 5
2.4 加载标定 5
2.5 最小示例 5
2.6 使能与故障处理 6
2.7 运动控制 6
2.8 抓取与力控 7
2.9 读取状态 8
2.10 手动示教 9
2.11 标定 9
2.12 异常与错误处理 10

3 其他语言与生态 11
3.1 C++ SDK 11
3.2 ROS 2 与 MoveIt 2 11
3.3 仿真 12
3.4 上位机（LiteGrip Studio） 12
3.5 遥操作与轨迹回放 13

4 常见问题 13
4.1 Q&A 13

5 附录一：CAN 通信协议 14
5.1 物理层与帧类型 14
5.2 MIT 控制帧 15
5.3 状态与错误码 16

5.4 参数寄存器 18
5.5 通信时序 19

6 附录二：连接排查 20
6.1 CAN 接口配置 20
6.2 连接排查 20

1 前言

English | 简体中文
LiteGrip 是 NEXFORM ROBOTICS 面向科研教育、AI 机器人开发与轻量级工业自动化场景设计的自适应两指平动夹爪，有效行程 85.000 mm。夹爪通过 USB-CAN 适配器与上位机通信（经典 CAN，1 Mbps），配套 Python SDK、C++ SDK、ROS 2 驱动与仿真模型。
本文档面向集成工程师，说明 SDK 的安装与调用、抓取与标定的用法、异常处理、生态组件，以及精简的 CAN 协议附录。产品规格、安全须知与选型方法见《产品手册》；安装、接线与首次运行见《使用教程》。

1.1 软件组成与依赖

组件	仓库	作用
Python SDK	litegrip-python	主力接口，本文档的主体
C++ SDK	litegrip-cpp	不依赖 ROS 的 C++17 接口
ROS 2 驱动	litegrip-ros2	`ros2_control` 硬件接口
MoveIt 2 配置	litegrip-moveit2	夹爪的规划与演示
模型	litegrip-description	URDF 与网格
仿真	litegrip-pybullet、litegrip-mujoco、litegrip-isaacsim	模型在环验证
上位机	litegrip-studio	图形化操作台

Python SDK 的运行环境：

项	要求
操作系统	仅 Linux，内核需支持 SocketCAN
架构	x86_64 或 arm64
Python	3.8 及以上
运行时依赖	无，只用 Python 标准库与 SocketCAN
可选依赖	`eclipse-zenoh>=1.0`，仅 zenoh 遥操作链路需要

项	要求
硬件	USB-CAN 适配器、24 V 电源（夹爪内置 120 Ω 终端电阻）

Python SDK 与各仿真包都没有发布到 PyPI，请从源码检出安装。

1.2 单位说明

参数	单位	说明
开口 / 位置	mm	毫米，两指内侧面之间的真实距离
电机角度	rad	弧度
速度	mm/s、rad/s	毫米每秒、弧度每秒
力矩	N·m	牛顿米
夹持力	N	牛顿
温度	°C	摄氏度

SDK 位置 p 与本文档的 “开口” 是同一个量：全闭为 0，全开为 85.000 mm，单指行程 42.500 mm。它不是电机角度，也不是单指位移，而是卡尺在两指指尖之间量到的数。

> **注意**：电机角度与标定端点因台而异，本文档不写这些 rad 数值。需要时请读本机标定文件，或直接使用毫米接口。

1.3 安全须知

1.3.1 简介

本节说明通过软件驱动夹爪时必须遵守的安全原则。夹爪运动时会夹持、碰撞，断电后还会松脱；使用人员应充分认识操作风险，并经过培训后使用 SDK 开发与调试。

1.3.2 安全警示标志

本文档使用以下警示级别，产品上的图形标识为准：

- 危险：可能导致人员伤亡或设备严重损坏的危险情况。
- 警告：如不避免，可能导致人员伤亡或设备严重损坏的危险情况。
- 高温：存在高温部件，接触可能造成灼烫伤害或设备损坏。
- 注意：如不避免，可能导致人员伤害或设备损坏的一般警示。

1.3.3 安全注意事项（软件侧）

1. **软件不是安全限位**。位置目标超出标定行程会被静默钳位到端点，kp、kd、dq 越界会被静默饱和，调用方收不到任何提示。“命令不越界” 必须由集成方在应用层实现。
2. **必须配置硬件急停回路，且其动作不依赖 CAN 通信与上位机软件**。进程卡死、CAN 掉线或 Python 异常时，软件侧无法停机。
3. **必须配置机械挡块**作为最后一道防线。标定端点不是硬限位。
4. **不要把 `stop()` 或 `disable()` 当作安全停止**。 `stop()` 只发一帧零力矩，电机保持使能且可被反驱；`disable()` 的返回值只表示命令已发出，没有得到反馈证实。
5. **不要把函数的返回值当作动作已完成的证据**。运动接口的返回值只表示帧已发出，只有反馈帧能证明实际状态。
6. **力标定完成前不要使用任何力参数**。 `force_n` 按名义系数换算，不是标定值，不能用于安全决策或负载设计。
7. **断开 24 V 后再做安装、维护或清洁**。
8. 调试与标定必须有人在场监护，硬件急停保持可用。发现异响、卡顿、异常温升或位置跳变时，立即停机并切断 24 V。
9. **驱动器在 0.4 s 内收不到主机的帧会自动失能**。控制器必须周期性发帧；关闭该保护后，掉线时电机会保持最后一条力矩指令，未经评估不要关闭。

1.3.4 责任及规范

夹爪可与其他设备组成完整系统，本文档不包含完整系统的设计、安装与操作。完整系统的安全性取决于集成方式，使用者需依据所在国家或地区的法律法规与安全规范完成风险评估，并采取相应防护措施。
开始使用本夹爪即视为已阅读、理解并接受本文档及安全信息的全部条款。使用者对自身行为及后果负责，仅将夹爪用于正当目的。对违反使用要求或不可抗力导致的人身伤害、事故或财产损失，我司不承担责任。

1.3.5 风险评估

从软件侧看，风险评估应特别考虑以下失效模式：

失效模式	后果	缓解措施
上位机进程卡死	机构保持最后一条指令	驱动器通信超时，加硬件急停
CAN 掉线	软件无法停机	驱动器超时自动失能，加硬件急停

失效模式	后果	缓解措施
位置读数失效	闭环判断错误	上位机自行校验读数，加硬件急停
目标计算错误	机构撞向端点	应用层限幅，加机械挡块
多个主站并发	指令互相覆盖	保证单一主站，软件层互斥

使用者须通过风险评估判断相关危险是否构成不可接受的风险，并采取相应措施。夹爪不适合无专业人员指导且不具备完全民事行为能力的人士使用。

2 SDK 使用指南

2.1 安装 SDK

Python SDK 没有发布到 PyPI，安装从检出仓库开始：

```
git clone https://github.com/nexform-tech/litegrip-python.git
cd litegrip-python
python3 -m pip install .
python3 -c "import litegrip; print(litegrip.__version__)"
```

安装成功后最后一条命令会打印版本号。直接从源码运行而没有安装时，它会打印 `0.0.0+source`，这是 “从源码导入” 的标记，不是错误。
不想安装时，把 PYTHONPATH 指向源码树即可：

```
PYTHONPATH=/path/to/litegrip-python/src python3 your_script.py
```

> **注意**：驱动真机的那台机器请用 `pip install .`，不要用 `pip install -e .`。可编辑安装会把解释器指向工作副本，一次 `git checkout` 就会悄悄换掉实际运行的代码。

2.2 配置 CAN 接口

接口必须存在、处于 up 状态，并使用经典 CAN、1 Mbps。配置需要 root 或 CAP_NET_ADMIN 能力。

```
ip -details link show can0
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 fd off
sudo ip link set can0 up
```

> **注意**：`fd off` 不能省略。接口被配成 CAN FD 模式时，即使波特率正确也完全无法通信，现象和 “没有接 24 V” 很像。

没有硬件时，可以用虚拟总线验证软件环境：

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
```

虚拟总线上没有电机，`enable()` 重试耗尽后会返回 `ok=False`，这是预期行为。

2.3 接 24 V 电源

电机要出力就需要 24 V，请在第一条运动命令之前接通。

> **注意**：CAN 通信由 USB-CAN 适配器独立供电，所以不接 24 V 时仍能读状态和参数，但电机报欠压故障（`ERR = 0x9`）且不响应运动命令。排查 “命令发了但不动” 时，先查 24 V。

2.4 加载标定

每一台的行程端点都不同，运动之前必须先加载标定：

```
gripper.connect()
gripper.load_calibration()   # SDK 内置的出厂标定
gripper.enable()
```

> **注意**：`connect()` 不会加载标定。没有标定时，`open()`、`close()`、`grasp()` 以及每个定位接口都抛 `CommandError`，而不是跑在占位系数上。

`load_calibration()` 需要指定本机文件时传 `path=`，需要声明安装方向时传 `template="normal"` 或 `template="reverse"`。两者不能同时传。

2.5 最小示例

```
from litegrip import LiteGrip

with LiteGrip(channel="can0", can_id=0x08) as gripper:
    gripper.load_calibration()
    gripper.enable()

    gripper.open()                                      # 50 mm/s；随包标定停在 80 mm
    gripper.close()                                     # 顶到闭合侧限位
    result = gripper.grasp(force_n=20.0, hold_s=3.0)    # 闭合到夹住，再保持 20 N
    print(result.ok, result.reached, result.stalled, result.cycles)
```

运行 `python3 first_move.py`。夹住物体时典型输出是 `True False True 12`：

字段	含义
ok	保持阶段正常结束，这是应该看的字段
reached	到达目标角度且没有堵转，也就是空夹合拢
stalled	位置停止变化，夹住物体时就是这个结果
cycles	保持阶段跑了多少个周期

注意：成功的出力抓取是 stalled=True、reached=False，不要指望 reached=True。夹住物体和空夹到限位在堵转判据上完全一样，要区分请读 get_position()：空夹停在闭合端点，夹到物体则停在端点之前。

2.6 使能与故障处理

```
gripper.enable()        # 使能并等待反馈确认
gripper.disable()       # 失能，力矩归零
gripper.clear_fault()   # 清除驱动器故障
gripper.stop()          # 发一帧零力矩
```

enable() 先清除已锁存的故障，再使能，然后等待一帧反馈确认 ERR == 0x1。重试耗尽时它返回 EnableResult(ok=False)，不抛异常，所以要检查 ok。

方法	说明
enable(retries=None)	使能并确认；retries=None 时取 MotionConfig.enable_retries（3）
disable()	失能；返回值只表示命令已发出
clear_fault()	失能、清故障、再使能并确认
stop()	急停一帧零力矩；不使能也不失能，不锁存

注意：stop() 不是安全级停止。它发完一帧就返回，电机保持使能并接受下一条命令。真正的急停要用硬件开关切断 24 V。

2.7 运动控制

方法	作用
`open()`	开向 `work_stroke_mm` 或张开端点；随包标定停在 80 mm
`close()`	顶到闭合侧限位，不接力量参数
`grasp()`	闭合到堵转再保持力，见下一节
`goto(position_mm)`	走到绝对位置（mm）
`goto_rad(position_rad)`	走到绝对位置（rad）
`move_to(target_rad)`	`goto_rad()` 的长时长版本
`move_at_speed(target_mm, speed_mm_s=30.0)`	按给定线速度运动
`move_at_speed_rad(target_rad, speed_rad_s=0.5)`	按给定角速度运动
`home()`	走回标定的闭合位置

```python
with LiteGrip(channel="can0", can_id=0x08) as gripper:
    gripper.load_calibration()
    gripper.enable()
    gripper.open()
    gripper.goto(60.0)
    gripper.close()
```

> **注意**：超出标定端点的目标会被静默钳位，既不抛异常也不提示。需要在越界时拒绝，请在调用前自行比较 `config.pos_closed_rad` 与 `config.pos_open_rad`。
>
> **注意**：`duration` 只决定这一串帧持续多久，不是速度上限。要限制速度请用 `move_at_speed()` 或 `move_at_speed_rad()`。

2.8 抓取与力控

```
grasp(force_n=None, hold_s=0.0, *, progress=None) -> GraspResult
```

`force_n=None` 时取 `MotionConfig.force_n`（20.0 N）。`hold_s=0.0` 表示一直保持，直到出故障、失控看门狗触发或按下 Ctrl+C。

`grasp()` 先朝物体闭合，检测到位置停止变化后进入保持阶段；保持帧没有位置和速度增益，唯一的输出是 `force_n × 0.1` N·m 的前馈力矩。

GraspResult 字段	含义
ok	保持阶段正常结束
reached	到达目标角度且没有堵转，即空夹合拢
stalled	位置停止变化，夹住物体时的正常结果
cycles	保持阶段跑过的周期数，0 表示没有进入保持
travel_guard	失控看门狗触发并卸掉了力矩
state	结束时的 GripperState

注意：force_n 没有做过力标定。0.1 N·m/N 是名义系数，force_n=0 表示零前馈力矩，不是位置保持。力标定完成前，不要把这个参数用于安全决策或负载设计。

注意：没有 “在当前位置施加一个力” 的接口，这是刻意的。保持帧没有位置误差，夹口之间没有东西时力矩就是唯一的力，机构会一路加速。需要出力请用 grasp()。

2.9 读取状态

```
state = gripper.get_state(wait=True)
print(state.position_mm, state.temperature_mos, hex(state.error_code))
```

字段	说明
position_mm	实际开口（mm）
position_rad	电机角度（rad）
velocity_rad_s	速度
torque_nm	力矩
force_n	估算夹持力，未标定
temperature_mos / temperature_coil	MOS 温度 / 线圈温度
error_code	错误码，0 表示失能，1 表示使能
fresh / data_age_s	本次读取是否有帧应答 / 距那帧过了多久

wait=True 最多等 50 ms 收一帧新反馈；wait=False 直接返回缓存，适合高频控制循环。

注意：反馈是问询式的，电机只在收到主机的帧之后才回帧，单纯监听总线永远收不到任何反馈。

被使能但静止的夹爪什么都不发，此时 `get_state(wait=True)` 会把 50 ms 等满，然后交回上一帧。

注意：`force_n` 由 `torque_nm × 10` 折算，10.0 是名义系数，不是标定值。`aperture_mm` 已经是完整开口，与 `position_mm` 相同，不要再乘 2。

2.10 手动示教

```
gripper.enter_zero_gravity()        # 夹爪变软，可以用手推动
input(" 摆好位置后按回车……")
gripper.exit_zero_gravity()         # 以当前位置为目标保持
```

`enter_zero_gravity()` 让电机保持使能，但持续发 `kp=0, kd=0, tau=0` 的帧，机构可以被自由推动。

注意：零重力模式不保持位置，松手后夹口停在哪里由重力和摩擦决定。要 “松手即停在原处”，必须调用 `exit_zero_gravity()`。退出时不校验位置，手推超出行程也会被直接保持。

注意：`enter_zero_gravity(duration=...)` 在计时结束后只是停止发帧，不会自动回到位置保持；控制器必须继续发帧，否则驱动器会在 0.4 s 后失能。

2.11 标定

方法	驱动力	限位如何检测	需要人
`zero()`	电机	堵转检测，一次走完两端	需在场
`calibrate()`	电机	堵转检测	需在场
`calibrate_guided()`	电机	每个端点按回车确认，或自动堵转检测	每个端点确认
`calibrate_manual()`	手推（零重力）	手推到底，记录极值	全程手推

`zero()`、`calibrate()` 与 `calibrate_guided()` 会一路顶进机械硬限位，中途不会自行停止。运行前确认结构受得住这股力，并且现场有人守着硬件急停。推荐 `calibrate_manual()`，它不给机构施加任何电机力。标定结果用 `save_calibration()` 保存：

方法	说明
`save_calibration(path=None)`	默认写到 `~/.litegrip/<channel>_calibration.json`，返回写入路径
`load_calibration(path=None, template=None)`	把标定加载进 `config`

方法	说明
`load_template("normal")` / `load_template("reverse")`	加载安装方向模板

注意：`load_calibration()` 不校验文件内容，只处理文件不存在与 JSON 解析失败。更换夹爪、电机或手指，发生碰撞或过载之后，必须重新标定。

2.12 异常与错误处理

异常	触发条件
`LiteGripError`	基类，可携带 `error_code`
`CommError`	CAN 总线读或写失败
`ConnectError`	CAN 接口不可用，或电机不应答
`CommandError`	未标定、行程为零、命令帧发不出去，或参数组合非法
`CANTimeoutError`	总线上没有响应
`HardwareError`	驱动器报告欠压、过流、过温等故障
`NotInitializedError`	需要已连接或已使能的接口在条件不满足时被调用

```
from litegrip import LiteGrip, HardwareError, CommandError, LiteGripError

try:
    with LiteGrip(channel="can0", can_id=0x08) as gripper:
        gripper.load_calibration()
        gripper.enable()
        gripper.grasp(force_n=20.0, hold_s=3.0)
except CommandError:
    print("calibration missing or command rejected")
except HardwareError as error:
    print(f"driver fault: {error}")
except LiteGripError as error:
    print(f"SDK error: {error}")
```

注意：钳位与饱和不会抛异常。越界目标被静默钳位，越界的 kp、kd、dq 被静默饱和，`force_n`

的数值范围不校验。非有限值以及超出型号量程的 `tau` 会抛 `CommandError`。只捕异常的程序发现不了钳位。

3 其他语言与生态

3.1 C++ SDK

litegrip-cpp 是与 ROS 无关的 C++ SDK：C++17、CMake 3.16 或更新、Linux 与 SocketCAN, 除标准库、`pthread` 和 SocketCAN 头文件外没有第三方依赖。

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure
cmake --install build --prefix "$HOME/.local"
```

测试不需要硬件、CAN 接口或 root。在自己的工程中使用：

```
find_package(litegrip_cpp REQUIRED)
target_link_libraries(my_app PRIVATE litegrip_cpp::litegrip_cpp)
```

> **注意**：在安装前缀之外构建的程序需要 `LITEGRIP_DATA_DIR="$HOME/.local/share/litegrip_cpp"` 才能找到标定数据，否则 `load_calibration()` 会打印试过的路径并返回 `false`。

C++ SDK 的接口与 Python 基本对应，但有几处差异：

- 轨迹录制与回放、遥操作、`write_zero()` 只有 Python 提供。
- 两个 SDK 都没有 `set_force()`，要出力请用 `grasp()`。
- `close()` 收速度与进度回调，不接收力。
- 命令速度上限由 `kMaxCommandVelocityCeilingRadS`（1.5 rad/s）限制，只能调低。

3.2 ROS 2 与 MoveIt 2

litegrip-ros2 提供 `ros2_control` 的 `SystemInterface` 插件，直接由 C++ SDK 支撑，链路里没有 Python 守护进程。运动由一个关节 `gripper_opening_joint` 承载，两根手指是它的 mimic。

参数	默认值	说明
`channel` / `can_id`	`can0` / `0x08`	总线接口与电机 ID
`dry_run`	`true`	为 `true` 时不发运动帧
`hardware_enable`	`false`	必须为 `true` 才会驱动电机
`max_velocity_rad_s`	`1.5`	命令速度上限，只能调低

参数	默认值	说明
max_feedback_velocity_rad_s	-1.0	最坏反馈速度界；-1.0 表示未给出，此时拒绝运动
torque_limit_nm	3.5	控制力矩总预算

只有 dry_run=false 且 hardware_enable=true 才会发出运动帧。max_feedback_velocity_rad_s 必须在你的机器上标定，-1.0 是刻意的失效安全行为。

```
ros2 launch litegrip_moveit_config demo.launch.py
ros2 launch litegrip_moveit_config demo.launch.py \
  dry_run:=false hardware_enable:=true \
  max_feedback_velocity_rad_s:=0.9
```

模型在 litegrip-description 中，可用 ros2 launch litegrip_description display.launch.py 查看。

注意：litegrip-ros1 是空仓库，没有源码、包或发布。不要围绕它规划 ROS 1 集成。

3.3 仿真

包	入口	提供什么
litegrip-pybullet	python3 examples/01_hello_sim.py --headless	GripperSim，以及 sim→real 与 real→sim 示例
litegrip-mujoco	from litegrip_mujoco import MujocoGripper	MujocoGripper、MirrorMode、DualGripper
litegrip-isaacsim	ISAAC_SIM_PATH=... ./run_gripper.sh	Isaac Sim 节点，以及连到真机 CAN 的桥

两个 Python 仿真环境都使用 litegrip-description 里的同一套模型。三者都没有发布到 PyPI，请从源码检出安装。

3.4 上位机（LiteGrip Studio）

litegrip-studio 是一个 PyQt5 操作台，用来驱动、标定和监视夹爪，需要 Python 3.10 或更新，以及 PyQt5、pyqtgraph、numpy。

```
./run_litegrip_studio.sh sim        # 内置仿真夹爪
```

```
./run_litegrip_studio.sh gui        # can0 上的真机
./run_litegrip_studio.sh selftest   # 自检，不需要 Qt 与硬件
./run_litegrip_studio.sh test       # 测试套件
```

SDK 内置在 `src/litegrip` 下，克隆后不需要额外安装。凡是要脚本化的任务，仍然走 SDK。

3.5 遥操作与轨迹回放

Python SDK 提供 `GripperTeleop` 做主从遥操作，线上传的是归一化到 `[0, 1]` 的开口，两端不必共享标定。轨迹录制用 `TrajectoryRecorder`，回放用 `TrajectoryPlayer`。示例脚本是 `examples/teleop.py`（每端一个进程）与 `examples/trajectory.py`（录制、查看与回放）。

> **注意**：`TeleopTransport` 可以自行实现 pub、sub、close 来接入自己的链路；不要在遥操作与轨迹回放同时运行时共用同一台夹爪。

4 常见问题

4.1 Q&A

问：SDK 支持哪些平台？答：仅 Linux，需要内核支持 SocketCAN。SDK 只用 Python 标准库，没有运行时依赖。
问：`pip install litegrip` 能用吗？答：不能。本包没有发布到 PyPI，请克隆仓库后执行 `python3 -m pip install .`。
问：为什么配置 CAN 时一定要加 `fd off`？答：本产品用经典 CAN。按 CAN FD 配置会完全无法通信，现象和 “没接 24 V” 很像。
问：为什么我监听总线什么都收不到？答：反馈是问询式的，电机只在收到主机发来的帧之后才回帧，必须先发帧。
问：连接之后要做什么？答：先 `load_calibration()`，再 `enable()`。`connect()` 不会自动加载标定。
问：`open()` 抛 `CommandError` 说配置未标定，是哪里不对？答：标定没加载。先调 `load_calibration()`，或跑一次 `zero()`。
问：`goto()` 去的位置不对，却不报错，为什么？答：超出标定端点的目标会被静默钳位，这是设计行为。需要在越界时拒绝，请自己先比较端点。
问：函数返回 `True`，是不是就成功了？答：看是哪个函数。`enable()` 的 `True` 有反馈帧背书；`disable()` 的 `True` 只表示命令已发出；运动接口的返回值只表示帧发出去了，到位与否要读 `get_state()`。
问：为什么 `close()` 不按我设的力停住？答：`close()` 没有力参数。只有 `grasp()` 接收 `force_n`，而且这个参数没有做过力标定。
问：`grasp()` 返回 `stalled=True`，是夹到东西了吗？答：不一定。空夹到限位也会堵转，请读 `get_position()`：空夹停在闭合端点，夹到物体则停在端点之前。
问：`grasp()` 返回 `ok=False`，是坏了吗？答：不一定。ok 只报保持阶段是否正常结束，要结合 `reached`、`stalled` 与 `cycles` 一起读。

问：手指停在行程之外了，怎么办？答：用 `goto_rad()` 开回来，它会把目标钳到标定端点之内。如果机构卡死，先手工排除干涉，不要继续施力。
问：零重力模式下松手，手指会掉吗？答：会动。零重力模式不出力，要停在原处请调用 `exit_zero_gravity()`。
问：位置读数和卡尺量出来对不上？答：若差一个固定百分比，是标定刻度错了；若差一个固定偏移量，是测量基准面不同。位置读数就是两指指尖之间的开口。
问：可以多台上位机同时控制吗？答：不能。必须保证单一 CAN 主站，并在软件层做互斥。一个进程驱动多台夹爪请用 `litegrip.can.controller`。
问：技术支持渠道是什么？答：NEXFORM ROBOTICS 技术团队。联系方式见随产品交付的文档与销售渠道。

5 附录一：CAN 通信协议

本附录供不使用 SDK、或需要在其他语言中自行实现控制的集成工程师参考。SDK 的位置钳制、零力矩停机等处理不会自动生效，越界判断、限速、急停回路与周期性发帧都需要自行实现。

5.1 物理层与帧类型

项目	数值
总线类型	经典 CAN（CAN 2.0）
波特率	1 Mbps
CAN FD	不使用，必须关闭
数据帧长度	8 字节
终端电阻	内置 120 Ω，无需外接

名称	数值	含义
电机 CAN ID（`ESC_ID`）	`0x08`	主机发往电机的控制帧与命令帧
反馈 ID（`MST_ID`）	`0x18`	电机发往主机的帧
广播 ID	`0x7FF`	状态刷新帧与参数帧，目标电机 ID 写在数据里

帧类型	方向	仲裁 ID	用途
MIT 控制帧	主机 → 电机	`0x08`	位置、速度、刚度、阻尼、前馈力矩
命令帧	主机 → 电机	`0x08`	使能、失能、清除故障、设置零点
状态刷新帧	主机 → 电机	`0x7FF`	请求一帧状态，不改变输出
参数帧	主机 → 电机	`0x7FF`	读写寄存器与保存到 Flash
反馈帧	电机 → 主机	`0x18`	状态、位置、速度、力矩、温度、错误码

命令帧的字节 0 到 6 全部为 `0xFF`，字节 7 为命令码：

命令	命令码	完整数据字节
使能	`0xFC`	`FF FF FF FF FF FF FF FC`
失能	`0xFD`	`FF FF FF FF FF FF FF FD`
清除故障	`0xFB`	`FF FF FF FF FF FF FF FB`
设置零点	`0xFE`	`FF FF FF FF FF FF FF FE`

状态刷新帧发往 `0x7FF`, 数据为 `[can_id 低字节, can_id 高字节, 0xCC, 0x00]`；电机 CAN ID 为 `0x08` 时即 `08 00 CC 00`。

> **注意**：只有状态刷新帧与参数帧用广播 ID `0x7FF` 寻址。MIT 控制帧与命令帧靠仲裁 ID 寻址，要发到 `0x08`。发错地址时没有电机会应答或执行，现象与 “没接 24 V” 很像。
>
> **注意**：“设置零点” 会把当前位置记为机械零点，且不可撤销。夹爪的零点靠标定得到，不要发送这条命令。

5.2 MIT 控制帧

字段	位宽	量程	说明
位置 q	16 位	±12.5 rad	目标角度
速度 dq	12 位	±30 rad/s	目标速度
位置刚度 kp	12 位	0 ~ 500	越大越硬
阻尼 kd	12 位	0 ~ 5	阻尼系数

字段	位宽	量程	说明
前馈力矩 tau	12 位	±10 N·m	前馈力矩

电机内部的力矩输出为：

```
tau_out = kp * (q_target - q_actual) + kd * (dq_target - dq_actual) + tau
```

字节排布：

字节	内容
0	q[15:8]
1	q[7:0]
2	dq[11:4]
3	dq[3:0] 在高 4 位，kp[11:8] 在低 4 位
4	kp[7:0]
5	kd[11:4]
6	kd[3:0] 在高 4 位，tau[11:8] 在低 4 位
7	tau[7:0]

量化采用截断，不是四舍五入：

```
code = int((value - min) * (2^bits - 1) / (max - min))
value = code * (max - min) / (2^bits - 1) + min
```

因此解码值总是不大于命令值，误差落在 (-1 LSB, 0] 区间内。

> **注意**：零力矩帧里 q 字段仍然存在，但 kp = kd = 0 使位置项和速度项恒为零，q 填什么都不出力。SDK 在发零力矩帧时把 q 填成当前实测位置。
>
> **注意**：kp、kd 越界会被静默饱和到编码量程；非有限值或超出型号量程的 tau 会被拒绝。

5.3 状态与错误码

反馈帧的仲裁 ID 为 0x18：

字节	内容
0	`ERR[7:4]` 在高 4 位，`can_id[3:0]` 在低 4 位
1	`q[15:8]`
2	`q[7:0]`
3	`dq[11:4]`
4	`dq[3:0]` 在高 4 位，`tau[11:8]` 在低 4 位
5	`tau[7:0]`
6	MOS 温度（°C）
7	线圈温度（°C）

q、dq、tau 的解码规则与量程与 MIT 控制帧相同。

码值	含义	类别
`0x0`	已失能	状态
`0x1`	已使能	状态
`0x8`	过压故障	故障
`0x9`	欠压故障	故障
`0xA`	过流故障	故障
`0xB`	MOS 过温故障	故障
`0xC`	线圈过温故障	故障
`0xD`	通信丢失	故障
`0xE`	过载	故障

注意：`0x8` 与 `0x9` 很容易看反，顺序是 8 过压、9 欠压。

注意：这只是常用码。驱动器还会报告本表之外的故障，SDK 对未识别的码显示 “未知错误”，需要时请对照驱动器指示灯判读。

注意：判断使能状态必须用反馈帧。`ERR = 0x1` 才表示已使能；`ERR = 0x0` 时电机会接收位置指令但不输出力矩，表现为 “指令发了，机构不动”。

5.4 参数寄存器

参数帧一律发往 `0x7FF`：

操作	数据字节
读	`[can_id 低, can_id 高, 0x33, RID, 0, 0, 0, 0]`
写	`[can_id 低, can_id 高, 0x55, RID, d0, d1, d2, d3]`
保存	`[can_id 低, can_id 高, 0xAA, 0x01, 0, 0, 0, 0]`

响应帧结构相同，字节 2 回显操作码，字节 3 为 RID，字节 4 到 7 为数据。整数寄存器为小端无符号 32 位，浮点寄存器为小端 IEEE 754 单精度。

RID	名称	类型	说明
0	`UV_Value`	浮点	欠压保护阈值（V）
2	`OT_Value`	浮点	过温保护阈值（°C）
3	`OC_Value`	浮点	过流保护阈值（比例，非安培）
6	`MAX_SPD`	浮点	最大速度，真正的限速
7	`MST_ID`	整数	反馈帧 ID
8	`ESC_ID`	整数	电机 CAN ID
9	`TIMEOUT`	整数	通信超时，单位 50 μs
10	`CTRL_MODE`	整数	控制模式
14	`sw_ver`	整数	固件版本号
21	`PMAX`	浮点	位置映射范围（rad）
22	`VMAX`	浮点	速度映射范围（rad/s）
23	`TMAX`	浮点	力矩映射范围（N·m）
29	`OV_Value`	浮点	过压保护阈值（V）

示例：读 `TIMEOUT`（RID 9）发送 `08 00 33 09 00 00 00 00`；把 `TIMEOUT` 写为 8000（0.4 s）发送 `08 00 55 09 40 1F 00 00`；保存参数发送 `08 00 AA 01 00 00 00 00`。

注意：`OC_Value` 是比例值，`0.8` 表示 80%，不是 0.8 A。

注意：PMAX、VMAX、TMAX 是 MIT 帧字段的映射范围，不是保护阈值。真正的速度保护在 MAX_SPD。

注意：写入的参数断电后不会自动保留，必须显式发送保存帧，且保存前先失能电机。

5.5 通信时序

项目	数值	说明
使能成功判据	`ERR == 0x1`	必须用命令发出之后收到的状态帧
使能重试次数	3 次	全部失败时 `enable()` 返回 `ok=False`
失能验证	无	发完失能帧就返回
状态刷新等待	0.05 s	`get_state(wait=True)` 的等待时间
参数读取等待	0.5 s	`read_param()` 的默认超时
控制帧周期	5 ms（200 Hz）	运动过程中的节拍
驱动器通信超时	0.4 s	超时后自动退出使能

步骤	方向	帧
1	主机 → 电机	清除故障 `FF FF FF FF FF FF FF FB`
2	主机 → 电机	使能 `FF FF FF FF FF FF FF FC`
3	电机 → 主机	反馈帧，校验 `ERR = 0x1`
4	主机 → 电机	MIT 控制帧，持续发送
5	电机 → 主机	每收到一帧控制帧，回一帧反馈
6	主机 → 电机	零力矩帧
7	主机 → 电机	失能 `FF FF FF FF FF FF FF FD`
8	电机 → 主机	反馈帧，校验 `ERR = 0x0`

注意：第 3 步与第 8 步不能省略。命令帧是单向的，只有反馈帧能证明命令真正生效。

注意：驱动器在 0.4 s 内收不到主机的帧会自动退出使能，控制器必须保持周期性发送。

6 附录二：连接排查

6.1 CAN 接口配置

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 fd off
sudo ip link set can0 up
ip -details link show can0
```

无硬件时可用虚拟总线验证软件环境：

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
```

不经 SDK 的总线自检：

```
sudo apt install can-utils

# 1）被动监听：不发帧时应当什么都看不到
candump can0

# 2）另一个终端里发一帧状态刷新
cansend can0 7FF#0800CC00

# 3）读固件版本号（RID 14）
cansend can0 7FF#0800330E00000000
```

注意：candump 长时间没有输出不等于夹爪坏了。反馈是问询式的，先发第 2 步那一帧再判断。

最安全的使能自检是只接 USB-CAN、暂不接 24 V，然后连发 5 帧使能命令：

```
for i in $(seq 5); do cansend can0 08#FFFFFFFFFFFFFFFC; sleep 0.01; done
```

预期反馈的 ERR 为 0x9（欠压）。能收到 0x9 就证明总线通、电机在回话，问题被限定在供电侧。

注意：接通 24 V 之后再发使能帧，机构会真的运动。执行前确认夹指运动范围内无人、无干涉物。

6.2 连接排查

现象	可能原因	处理方法
完全收不到反馈帧	只监听未发送	反馈是问询式的，必须先发帧
	命令帧发到了 0x7FF	命令帧与 MIT 帧要发到 0x08
	CAN FD 未关闭	加 `fd off` 重新配置

现象	可能原因	处理方法
	外部重复接入了终端电阻	移除外部终端电阻，夹爪内置 120 Ω
使能无效，ERR 始终 `0x0`	24 V 未接通	接通 24 V
	利用了陈旧的反馈帧	必须用命令发出之后的帧判定
反馈 `ERR = 0x9`	24 V 未接通或电压不足	检查供电
发指令后机构不动，`ERR = 0x1`	目标位置与当前位置相同	确认目标值
	`kp = 0`	检查刚度参数
运动中突然失能	驱动器通信超时	保持周期性发送
	触发过温或过流保护	停机冷却，排查负载
反馈的 `tau` 略小于预期	截断量化	正常现象
多台设备互相干扰	多个主站并发	保证单一主站，软件层互斥

无法自行解决时，请记录产品型号与序列号、SDK 版本、Python 版本、操作系统与内核版本、CAN 接口配置、完整异常信息与堆栈，然后联系我们。

文档结束如有疑问或需要补充信息，请联系 NEXFORM ROBOTICS 技术团队。